The `link_to` method is one of Rails' built-in view helpers. It creates a hyperlink based on text to display and where to go - in this case, to the path for articles.
Let's add links to the other views as well, starting with adding this "New Article" link to `app/views/articles/index.html.erb`, placing it above the `<table>` tag:

```
<%= link_to 'New article', new_article_path %>
```

This link will allow you to bring up the form that lets you create a new article.

Now, add another link in `app/views/articles/new.html.erb`, underneath the form, to go back to the `index` action:

```
<%= form_for :article, url: articles_path do |f| %>
  ...
<% end %>

<%= link_to 'Back', articles_path %>
```

Finally, add a link to the `app/views/articles/show.html.erb` template to go back to the `index`action as well, so that people who are viewing a single article can go back and view the whole list again:

```
<p>
  <strong>Title:</strong>
  <%= @article.title %>
</p>

<p>
  <strong>Text:</strong>
  <%= @article.text %>
</p>

<%= link_to 'Back', articles_path %>
```

If you want to link to an action in the same controller, you don't need to specify the `:controller` option, as Rails will use the current controller by default.

In development mode (which is what you're working in by default), Rails reloads your application with every browser request, so there's no need to stop and restart the web server when a change is made.

5.10 Adding Some Validation

The model file, `app/models/article.rb` is about as simple as it can get:

```
class Article < ActiveRecord::Base
end
```

There isn't much to this file - but note that the `Article` class inherits from `ActiveRecord::Base`. Active Record supplies a great deal of functionality to your Rails models for free, including basic database CRUD (Create, Read, Update, Destroy) operations, data validation, as well as sophisticated search support and the ability to relate multiple models to one another.
Rails includes methods to help you validate the data that you send to models. Open the `app/models/article.rb` file and edit it:

```
class Article < ActiveRecord::Base
  validates :title, presence: true,
                    length: { minimum: 5 }
end
```

These changes will ensure that all articles have a title that is at least five characters long. Rails can validate a variety of conditions in a model, including the presence or uniqueness of columns, their format, and the existence of associated objects. Validations are covered in detail in Active Record Validations.
With the validation now in place, when you call `@article.save` on an invalid article, it will return `false`. If you open `app/controllers/articles_controller.rb` again, you'll notice that we don't check the result of calling `@article.save` inside the `create` action. If `@article.save` fails in this situation, we need to